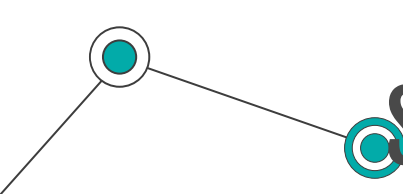




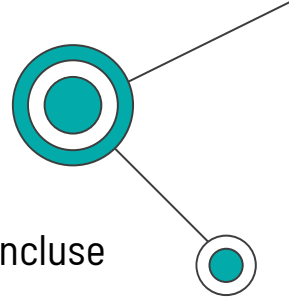
Laboratorio – Programmazione del calcolo quantistico

Franco Cirillo
University of Salerno
fracirillo@unisa.it

Tecnologie quantistiche per la sicurezza



Simulazione esatta e rumorosa



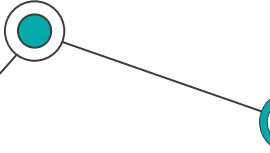
La simulazione esatta con Qiskit primitives mostra come utilizzare le primitive di riferimento incluse in Qiskit per eseguire la simulazione esatta di circuiti quantistici. (es. **StateVector**)

I processori quantistici attualmente esistenti sono affetti da errori, o rumore, per cui i risultati di una simulazione esatta non riflettono necessariamente i risultati che ci si aspetterebbe eseguendo i circuiti su hardware reale.

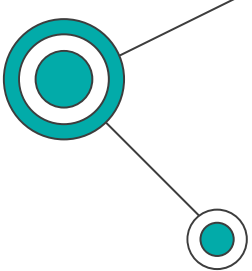
Mentre le primitive di riferimento di Qiskit non supportano il rumore di modellazione, **Qiskit Aer** include implementazioni delle primitive che supportano il rumore di modellazione.

Qiskit Aer è un simulatore di circuiti quantistici ad alte prestazioni che può essere utilizzato al posto delle primitive di riferimento per ottenere migliori prestazioni e maggiori funzionalità.

<https://qiskit.github.io/qiskit-aer/index.html>



Simulazione esatta e rumorosa



```
from qiskit.circuit.library import efficient_su2
from qiskit.quantum_info import SparsePauliOp
from qiskit.transpiler import generate_preset_pass_manager
from qiskit_aer import AerSimulator
from qiskit_aer.primitives import EstimatorV2 as Estimator
from qiskit_aer.noise import NoiseModel, depolarizing_error

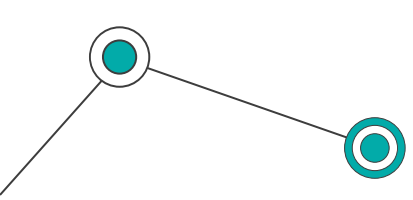
n_qubits = 8
circuit = efficient_su2(n_qubits)
circuit.draw("mpl")
observable = SparsePauliOp("Z" * n_qubits)
params = [0.1] * circuit.num_parameters

exact_estimator = Estimator()
# The circuit needs to be transpiled to the AerSimulator target
pass_manager = generate_preset_pass_manager(3, AerSimulator())
isa_circuit = pass_manager.run(circuit)
pub = (isa_circuit, observable, params)
job = exact_estimator.run([pub])
```

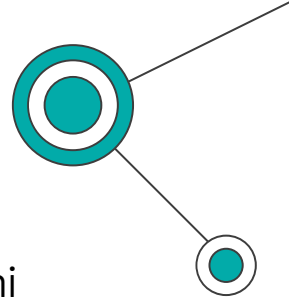
```
result = job.result()
pub_result = result[0]
exact_value = float(pub_result.data.evs)
exact_value
```

```
noise_model = NoiseModel()
cx_depolarizing_prob = 0.02
noise_model.add_all_qubit_quantum_error(
    depolarizing_error(cx_depolarizing_prob, 2), ["cx"])

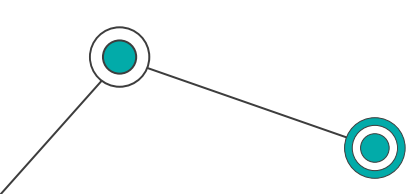
noisy_estimator = Estimator(
    options=dict(backend_options=dict(noise_model=noise_model)))
job = noisy_estimator.run([pub])
result = job.result()
pub_result = result[0]
noisy_value = float(pub_result.data.evs)
noisy_value
```



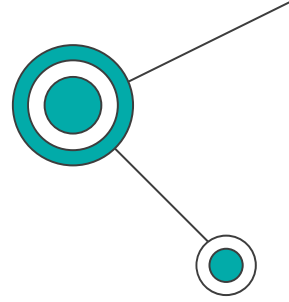
Esercizio



Eseguire il circuito precedente con il Sampler e mostrare i risultati nelle diverse visualizzazioni



Costruire modelli di rumore

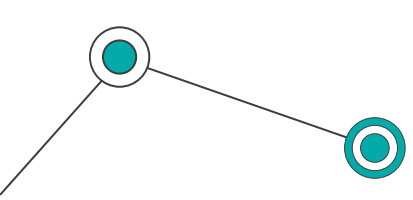


Il modulo Qiskit Aer noise contiene le classi Python per costruire modelli di rumore personalizzati per la simulazione. Le classi principali sono tre:

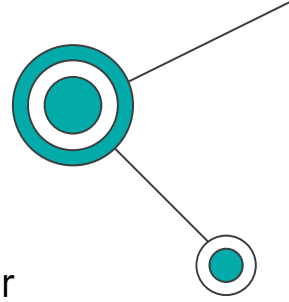
- La classe *NoiseModel* che memorizza un modello di rumore utilizzato per la simulazione del rumore.
- La classe *QuantumError* che descrive gli errori dei gate.
- La classe *ReadoutError* che descrive gli errori di lettura classici.

È possibile inizializzare un modello di rumore con i parametri impostati dai dati di calibrazione più recenti per un backend fisico. In questo modo si otterrà un modello di rumore che approssima gli errori che si incontrano quando si usa quel backend.

Se si desidera avere un controllo più dettagliato sui parametri del modello di rumore, è necessario creare il proprio modello di rumore



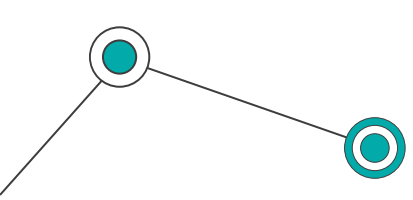
Problema di versioni



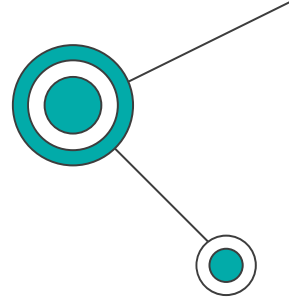
Attenzione alle versioni della documentazione e dei tutorial, molto spesso sono compatibili per versioni vecchie di qiskit.

<https://qiskit.github.io/qiskit-aer/>

<https://github.com/Qiskit/qiskit-aer>



Qiskit IBM Runtime



pip **install qiskit-ibm-runtime**

```
from qiskit_ibm_runtime import QiskitRuntimeService
```

```
QiskitRuntimeService.save_account(  
    token="<your-api-key>",  
    instance="<CRN>", # Optional  
)
```

```
service = QiskitRuntimeService()  
backend = service.least_busy(simulator=False,  
    operational=True)  
#real_backend = service.backend("ibm_fez")
```

```
pm =  
generate_preset_pass_manager(backend=backend,  
    optimization_level=1)
```

Simulazione

```
from qiskit_ibm_runtime.fake_provider import  
FakeBelemV2  
backend = FakeBelemV2()
```

<https://quantum.cloud.ibm.com/>



FakeAlgiers

FakeAlmadenV2

FakeArmonkV2

FakeAthensV2

FakeAuckland

FakeBelemV2

FakeBoeblingenV2

FakeBogotaV2

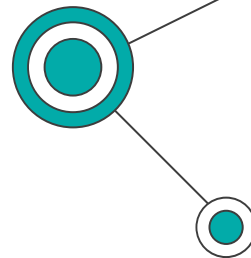
FakeBrisbane

FakeBrooklynV2

FakeBurlingtonV2

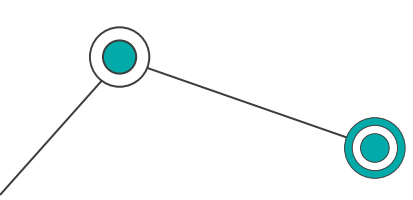
FakeCairoV2

Fake providers

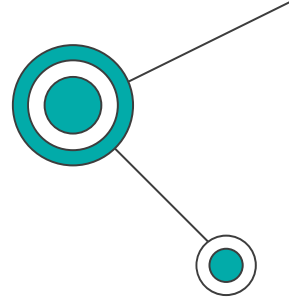


I **fake backend** sono costruiti per imitare i comportamenti dei sistemi IBM Quantum utilizzando le istantanee del sistema.

Le istantanee del sistema contengono informazioni importanti sul sistema quantistico, come la mappa di accoppiamento, le porte di base, le proprietà dei qubit (T1, T2, il tasso di errore, ecc.) che sono utili per testare il transpiler ed eseguire simulazioni rumorose del sistema.



Grover Algorithm

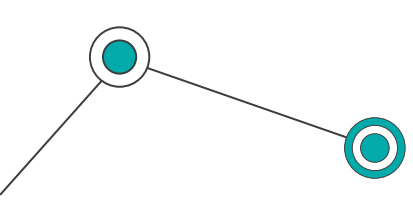


L'algoritmo di ricerca quantistica più noto è Grover's Algorithm, proposto nel 1996 dal ricercatore Lov Grover. Questo algoritmo permette di trovare una soluzione all'interno di uno spazio di ricerca non strutturato molto più velocemente rispetto a un algoritmo classico.

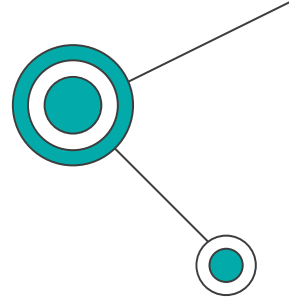
Mentre una ricerca classica richiede in media un numero di tentativi proporzionale a N , dove N è il numero di possibilità, l'algoritmo di Grover richiede circa $\sqrt{N}$ iterazioni.

Questo significa che il vantaggio diventa significativo quando il numero di possibili configurazioni è molto grande. Ad esempio, se esistono un milione di combinazioni possibili, un computer classico potrebbe dover eseguire fino a un milione di tentativi, mentre un computer quantistico potrebbe trovare la soluzione in circa mille iterazioni.

Questo non è un miglioramento esponenziale, ma rappresenta comunque un vantaggio quadratico molto importante per molti problemi pratici.



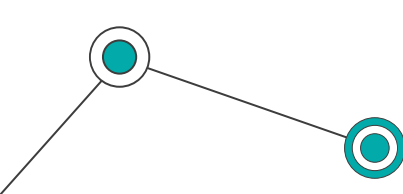
Implementazione



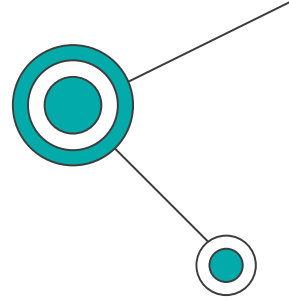
Per dimostrare questo concetto possiamo implementare una simulazione dell'algoritmo di Grover utilizzando la libreria **Qiskit**. Nel nostro esperimento utilizziamo un circuito quantistico con **4 qubit**.

Il primo passo consiste nel mettere tutti i qubit in sovrapposizione applicando una porta Hadamard a ciascuno di essi. Questo passaggio crea uno stato quantistico che rappresenta simultaneamente tutte le sedici configurazioni possibili. In altre parole, il sistema quantistico viene preparato in modo tale da contenere tutte le possibili soluzioni nello stesso momento.

```
from qiskit import QuantumCircuit  
  
qc = QuantumCircuit(4)  
qc.h([0,1,2,3])
```



Oracolo



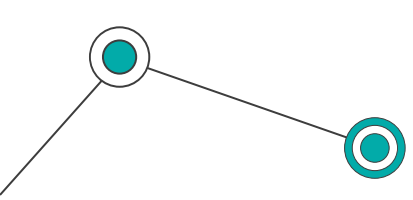
Una componente fondamentale dell'algoritmo di Grover è **l'oracolo**. L'oracolo è un circuito quantistico che ha il compito di riconoscere la soluzione corretta del problema.

In termini matematici, l'oracolo implementa una funzione che restituisce valore uno per la soluzione corretta e zero per tutte le altre configurazioni. Quando lo stato corretto attraversa l'oracolo, la sua fase quantistica viene invertita. Questo significa che lo stato viene moltiplicato per -1 .

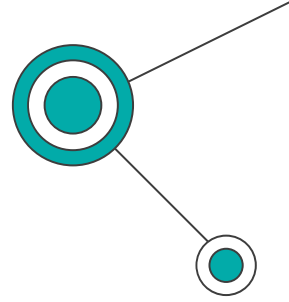
Questa modifica non cambia direttamente la probabilità di osservare lo stato durante la misura, ma è fondamentale per il funzionamento della fase successiva dell'algoritmo. L'inversione di fase infatti permette al diffusore di Grover di amplificare la probabilità della soluzione corretta rispetto a tutte le altre.

```
qc.x([2])  
qc.h(3)  
qc.mcx([0,1,2], 3)  
qc.h(3)  
qc.x([2])
```

$|1011\rangle \rightarrow -|1011\rangle$ altri stati invariati



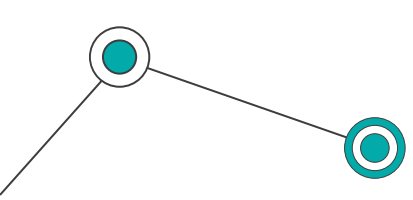
Oracolo



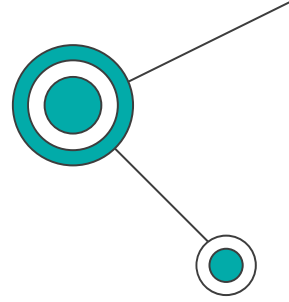
Quando si costruisce un oracolo bisogna trasformare una condizione logica del problema in un circuito quantistico.

Nel nostro esempio vogliamo riconoscere la configurazione 1011, cioè uno specifico stato dei quattro qubit. Il modo più semplice per farlo consiste nel trasformare temporaneamente la configurazione target nello stato 1111, perché è molto più facile costruire un circuito controllato che si attivi quando tutti i qubit sono uguali a uno.

```
1011
↓ X sui bit con valore 0
1111
↓ operazione controllata
  phase flip
↓ undo X
1011
```



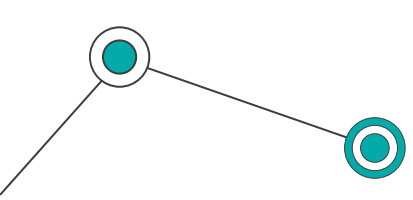
Oracolo



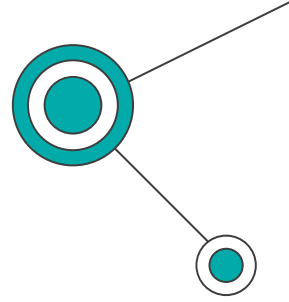
Per ottenere questa trasformazione si applica una porta X sui qubit che nel target hanno valore zero. La porta X infatti inverte il valore del qubit, trasformando zero in uno e uno in zero. In questo modo lo stato target viene temporaneamente convertito nello stato 1111.

Una volta ottenuto questo stato, è possibile utilizzare una porta controllata su più qubit per marcare la soluzione. Dopo aver eseguito questa operazione, le trasformazioni applicate all'inizio vengono invertite per riportare il sistema alla base originale.

1011
↓ X sui bit con valore 0
1111
↓ operazione controllata
phase flip
↓ undo X
1011



Oracolo

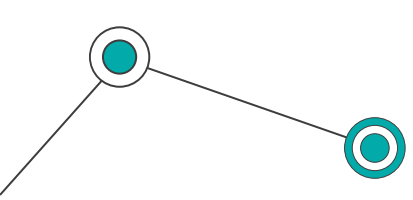


Nel circuito quantistico l'operazione principale utilizzata per implementare l'oracolo è una porta controllata su più qubit, chiamata Multi-Controlled X. Questa porta esegue una trasformazione sul qubit target solo se tutti i qubit di controllo sono nello stato uno.

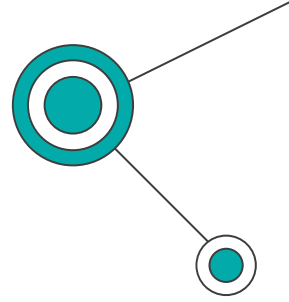
Tuttavia l'obiettivo dell'oracolo non è invertire un qubit, ma cambiare la fase dello stato soluzione. Per ottenere questo comportamento si utilizza un trucco molto comune nel quantum computing: applicare una trasformazione di Hadamard prima e dopo la porta controllata.

Le porte Hadamard trasformano infatti una operazione di tipo NOT in una operazione di inversione di fase. In questo modo il circuito riesce a cambiare la fase dello stato target senza alterare gli altri stati della sovrapposizione.

```
qc.x([2])  
qc.h(3)  
qc.mcx([0,1,2], 3)  
qc.h(3)  
qc.x([2])
```



Oracolo

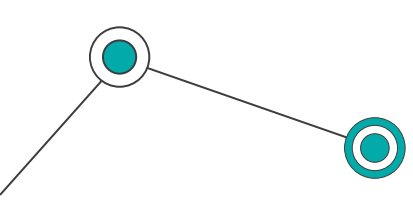


Per comprendere davvero cosa succede dopo l'oracolo bisogna osservare il comportamento delle ampiezze degli stati quantistici. Prima dell'applicazione dell'oracolo, dopo la sovrapposizione iniziale, tutte le configurazioni hanno la stessa ampiezza.

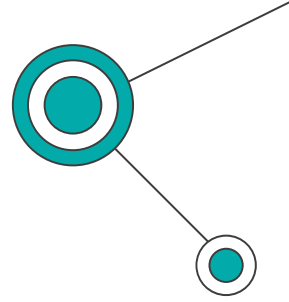
Questo significa che tutte le soluzioni sono equiprobabili. Quando viene applicato l'oracolo, l'ampiezza dello stato soluzione cambia segno, mentre tutte le altre restano positive.

Questo cambiamento non modifica immediatamente le probabilità di misura, ma crea una differenza di fase che può essere sfruttata nella fase successiva dell'algoritmo. Il passo successivo, cioè il diffusore di Grover, utilizza proprio questa differenza per amplificare la probabilità dello stato corretto attraverso interferenza costruttiva.

```
qc.x([2])  
qc.h(3)  
qc.mcx([0,1,2], 3)  
qc.h(3)  
qc.x([2])
```



Amplificazione

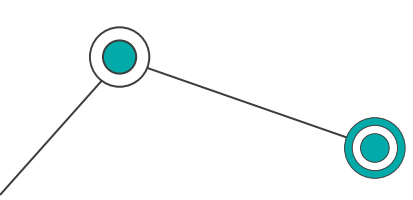


Dopo che l'oracolo ha marcato lo stato corretto invertendone la fase, l'algoritmo applica una trasformazione chiamata **diffusore** di Grover.

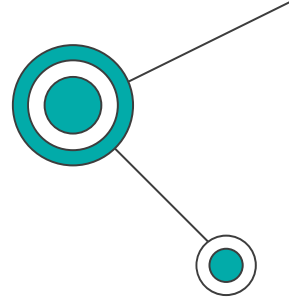
Questo circuito esegue un'operazione matematica nota come inversione rispetto alla media. In termini intuitivi, questa trasformazione aumenta la probabilità degli stati che sono stati marcati dall'oracolo e diminuisce la probabilità degli altri stati.

Ripetendo questa operazione alcune volte, la probabilità della soluzione corretta cresce progressivamente fino a diventare dominante rispetto a tutte le altre configurazioni. Questo processo è un esempio di interferenza quantistica controllata, in cui le ampiezze degli stati quantistici vengono manipolate in modo da rafforzare la soluzione corretta e cancellare gradualmente quelle sbagliate.

```
qc.h([0,1,2,3])  
qc.x([0,1,2,3])  
qc.h(3)  
qc.mcx([0,1,2],3)  
qc.h(3)  
qc.x([0,1,2,3])  
qc.h([0,1,2,3])
```

Amplificazione



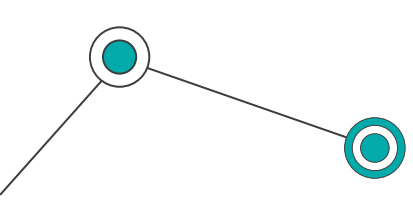
In termini intuitivi, si può immaginare che il diffusore prenda tutte le ampiezze degli stati quantistici e le rifletta rispetto al loro valore medio.

Il diffusore può essere implementato con una sequenza relativamente semplice di porte quantistiche. Il circuito esegue quattro operazioni principali:

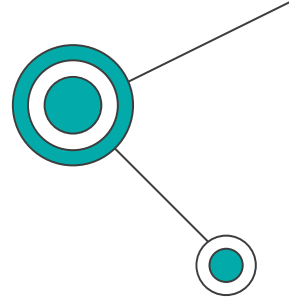
- Trasformare gli stati dalla base computazionale alla base della sovrapposizione tramite porte Hadamard.
- Invertire gli stati applicando porte X a tutti i qubit.
- Applicare un'operazione controllata che cambia la fase dello stato zero.
- Ripristinare il sistema applicando nuovamente le trasformazioni precedenti in ordine inverso.

```
qc.h([0,1,2,3])  
qc.x([0,1,2,3])  
qc.h(3)  
qc.mcx([0,1,2],3)  
qc.h(3)  
qc.x([0,1,2,3])  
qc.h([0,1,2,3])
```

Questa sequenza realizza esattamente la trasformazione matematica che riflette le ampiezze degli stati rispetto alla media.



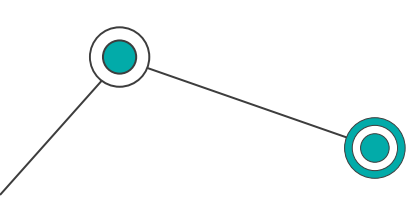
Amplificazione



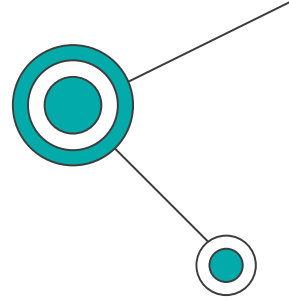
Un modo molto elegante per comprendere l'algoritmo di Grover è interpretarlo come una rotazione in uno spazio vettoriale. Lo spazio degli stati quantistici può essere visto come uno spazio bidimensionale generato da due vettori: uno che rappresenta lo stato soluzione e uno che rappresenta la sovrapposizione di tutti gli altri stati.

Ogni iterazione dell'algoritmo di Grover, composta da oracolo e diffusore, corrisponde a una rotazione del vettore di stato del sistema verso la direzione della soluzione. Dopo alcune iterazioni, il vettore si avvicina sempre di più allo stato corretto.

Quando la rotazione raggiunge il punto ottimale, la probabilità di osservare lo stato soluzione diventa molto elevata. Per questo motivo il numero ottimale di iterazioni dell'algoritmo è proporzionale alla radice quadrata del numero totale di configurazioni.



Misurazione



L'ultimo passo dell'algoritmo consiste nella misura dei qubit. La misura è l'operazione che trasforma lo stato quantistico in un risultato classico osservabile.

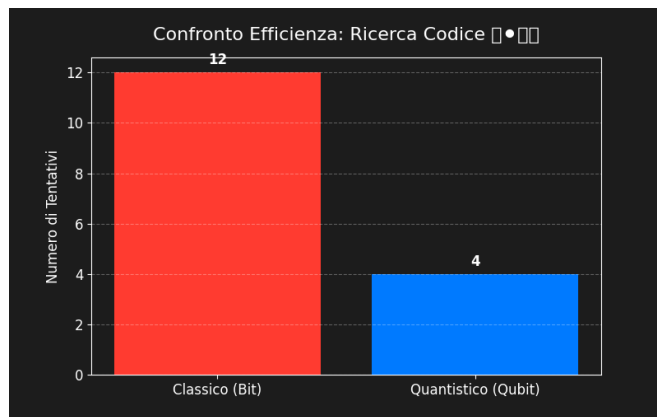
Quando misuriamo il sistema, la sovrapposizione quantistica collassa in uno degli stati possibili. Tuttavia, grazie all'amplificazione effettuata dal diffusore di Grover, la probabilità di ottenere la configurazione corretta è molto più alta rispetto a tutte le altre.

Di conseguenza, ripetendo l'esperimento più volte, il risultato che osserviamo più frequentemente è proprio la combinazione 1011, che rappresenta la soluzione del problema.

```
qc.measure_all()
```

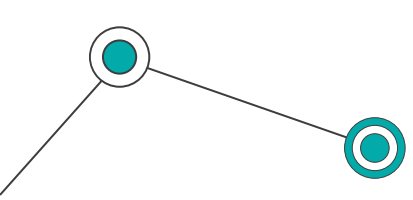
Confronto

Possiamo ora confrontare i due approcci. Il risultato è un esempio concreto di quantum speedup, cioè un miglioramento delle prestazioni ottenuto grazie all'utilizzo di algoritmi quantistici.

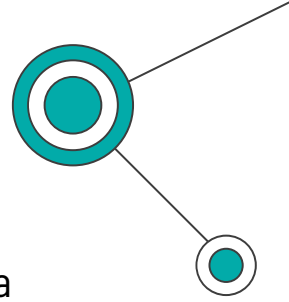


Media: $8 (n/2)$
Massimo $15 (n-1)$

$4 (\sqrt{n})$



Grover



Nonostante l'eleganza dell'algoritmo di Grover, nella pratica esiste una difficoltà importante: la costruzione dell'oracolo.

Per applicare Grover a un problema reale bisogna essere in grado di costruire un circuito quantistico che riconosca la soluzione corretta senza misurarla. Per problemi semplici, come l'esempio della combinazione a quattro bit, questo è relativamente facile. Tuttavia per problemi complessi, come la verifica di una formula logica o la risoluzione di un problema di ottimizzazione, costruire l'oracolo può richiedere circuiti quantistici molto grandi e complessi.

Per questo motivo gran parte della ricerca attuale nel campo del quantum computing riguarda proprio lo sviluppo di tecniche efficienti per costruire oracoli quantistici per problemi reali.



PENNYLANE

PennyLane (<https://pennylane.ai/>) is a cross-platform Python library for programming quantum computers. Its differentiable programming paradigm enables the execution and training of quantum programs on various backends.

PennyLane connects quantum computing with powerful machine learning frameworks like NumPy's autograd, JAX, and PyTorch, making them quantum-aware.

Program
quantum computers

Integrate with
machine learning

Master
quantum chemistry

The library comes with default simulator devices, but is well-integrated with external software and hardware to run quantum circuits—such as IBM's Qiskit, or Google's Cirq, Rigetti's Forest, or Xanadu's Strawberry Fields.



PENNYLANE

Device



`default.qubit`

A simple Python state vector simulator of qubit-based quantum circuits.

This is PennyLane's most featureful device, ideal for learning and prototyping.



`lightning.qubit`

A performant state vector simulator of quantum circuits.

This device is well-suited for 20+ qubit workflows, offering jit/qjit compatibility and more.



`qiskit.remote`

An entry point in PennyLane to access any Qiskit backend.

This device lets you access any device available through `qiskit_ibm_runtime`.



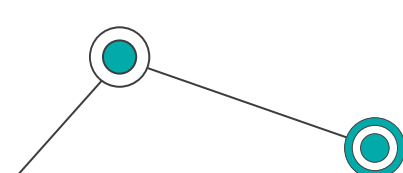
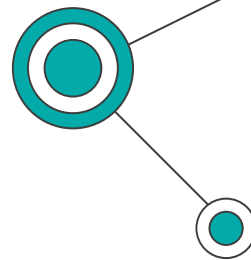
`default.mixed`

A simple mixed state simulator of qubit-based quantum circuits.

This device supports versatile noise models that are compatible with ML frameworks like JAX and Torch.



PENNYLANE



```
import pennylane as qml
```

```
from jax import numpy as np
```

```
import jax
```

```
dev1 = qml.device("lightning.qubit", wires=1)
```

```
@qml.qnode(dev1)
```

```
def circuit(params):
```

```
    qml.RX(params[0], wires=0)
```

```
    qml.RY(params[1], wires=0)
```

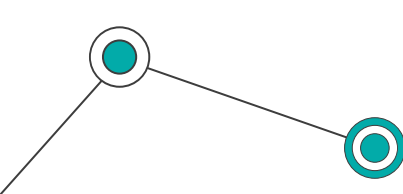
```
    return qml.expval(qml.PauliZ(0))
```

```
params = np.array([0.54, 0.12])
```

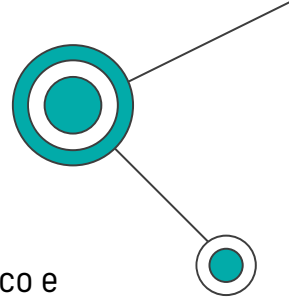
```
print(circuit(params))
```

- name: the name of the device to be loaded
- wires: the number of subsystems to initialize the device with (qubits)

Qnode è un circuito quantum legato ad un particolare device



Esercizio



In questo esercizio costruite un circuito quantistico a **3 qubit** usando PennyLane. Il circuito sarà parametrico e diviso in più "layer":

1. **Layer di rotazioni singole:** applicate su ciascun qubit una rotazione parametrica (RX, RY, RZ) con parametri $\theta_1, \theta_2, \theta_3$.
2. **Layer di entanglement:** collegate i qubit con CNOT per creare correlazioni tra loro.
3. **Secondo blocco parametrico:** applicate altre rotazioni sui qubit con nuovi parametri $\theta_4, \theta_5, \theta_6$.

Dopo aver definito il circuito come **QNode**, dovrete valutare:

- i valori attesi di PauliZ per ciascun qubit
- le probabilità complete sul sistema a 3 qubit (qml.probs)
- Provate a calcolare i risultati con:
 - tutti i parametri θ a 0
 - tutti i parametri θ a $\pi/4$
 - un vettore di parametri a scelta

Infine, osservate come cambiano le probabilità e i valori attesi, e commentate l'effetto dell'entanglement e dei parametri sul circuito.